

Smart Surveillance System

FastAPI, React/Vite, OpenCV and YOLO surveillance application with shared live video processing, tracking, loitering alerts, configurable zones, SQLite incident history, evidence images and optional Telegram notifications.

Architecture

Camera source -> one reconnecting camera worker -> one YOLO/tracking worker -> MJPEG /video + REST API -> React dashboard

The AI pipeline is shared: opening several dashboard tabs does not start additional YOLO workers. Screenshots belong in `outputs/` locally, or the Docker `surveillance_data` volume.

Requirements

- Windows 10/11: Python 3.11+, Node.js 18+, PowerShell. A local webcam is supported by the local backend.
- Docker: Docker Desktop for RTSP/network-camera or server deployment. Windows Docker webcam passthrough is not a supported workflow.
- An RTSP URL or a camera visible to OpenCV.

Windows setup and webcam run

From the repository root, install once:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass  
.\scripts\setup_windows.ps1
```

Use two PowerShell terminals:

```
.\scripts\start_backend.ps1
```

```
.\scripts\start_frontend.ps1
```

Equivalent backend command (the README does **not** use `python main.py`):

```
cd backend  
.\.venv\Scripts\Activate.ps1  
uvicorn api:app --host 127.0.0.1 --port 8001
```

The dashboard is normally at <http://localhost:5173> and Swagger API documentation is <http://127.0.0.1:8001/docs>.

Configuration

Copy `.env.example` to `.env` and edit it. `.env` is ignored by Git.

```
CAMERA_SOURCE=0
# CAMERA_SOURCE=rtsp://username:password@192.168.1.50:554/stream1
YOLO_MODEL=backend/yolov8n.pt
CONFIDENCE_THRESHOLD=0.5
TELEGRAM_ENABLED=false
CORS_ORIGINS=http://localhost:5173,http://127.0.0.1:5173
```

`CAMERA_SOURCE=0` (or `1`) selects a local camera. Numeric strings are converted to indexes; all other values are passed to OpenCV as RTSP URLs or video file paths. The source is redacted in `/status` so RTSP credentials are never returned. If unavailable, the API remains online and retries the camera periodically.

Set `TELEGRAM_ENABLED=true`, `BOT_TOKEN`, and `CHAT_ID` only when notifications are needed. Tokens and RTSP passwords must never be committed.

Docker / RTSP deployment

Set `CAMERA_SOURCE` and any Telegram values in `.env`, then run:

```
docker compose up --build -d
docker compose logs -f
```

Dashboard: <http://127.0.0.1:3000>. API: <http://127.0.0.1:8001/docs>.

In Docker, nginx exposes port 3000 and the frontend uses `VITE_API_BASE_URL=/api`; `backend:8001` remains Docker-only DNS and is never sent to browser JavaScript. Persistent SQLite data and captures are kept in the named `surveillance_data` volume.

Deployment paths

1. **Windows local webcam** — recommended for laptop/demo use. Run FastAPI natively with `CAMERA_SOURCE=0`; Docker Desktop is not a dependable Windows-webcam passthrough.
2. **RTSP/IP camera** — use an RTSP URL in `.env`; FastAPI reconnects without exiting when a stream is unavailable.
3. **Docker + RTSP** — compose puts nginx on port 3000. The browser uses same-origin `/api`; nginx forwards it to FastAPI and disables buffering for the MJPEG stream.
4. **Internet-facing** — requires a real TLS reverse proxy, `AUTH_ENABLED=true`, `PROTECT_MONITORING_ROUTES=true`, `COOKIE_SECURE=true`, strong secrets, restricted CORS, and firewall rules. This repository does not provide certificates or a public-domain deployment.

For a TLS terminator, use placeholder certificate paths only; do not commit keys:

```
server {
  listen 443 ssl;
  server_name surveillance.example.com;
  ssl_certificate /etc/nginx/certs/fullchain.pem;
  ssl_certificate_key /etc/nginx/certs/privkey.pem;
  # Proxy / to the compose nginx service; retain its /api and /video rules.
}
```

Nginx applies a small per-IP limit to `/api/auth/login` (5 requests/minute plus a five-request burst). For Internet exposure, pair it with edge/WAF rate limiting.

API

- `GET /health` — process health.
- `GET /ready` — API readiness plus camera/AI state.
- `GET /status` — configured (redacted) camera source, reconnect state, AI, CUDA and Telegram state.
- `GET /video` — MJPEG processed stream.
- `/incidents, /zones, /rules/loitering, /images` — dashboard data and configuration.

Route security matrix

Method	Route	Purpose	Auth when enabled	Mutates state
POST	<code>/auth/login</code>	Obtain admin JWT	No	No
GET	<code>/health, /ready, /status</code>	Operational monitoring	No	No
GET	<code>/video, /images, /outputs/*, /incidents, /zones, /rules/loitering</code>	Trusted-LAN surveillance viewing	No	No
PUT/DELETE	<code>/zones, /zones/{id}</code>	Zone administration	Admin JWT	Yes
PUT	<code>/rules/loitering</code>	Alert-rule administration	Admin JWT	Yes

Treat the read-only routes as surveillance-sensitive: restrict network access with a firewall/VPN or authenticated reverse proxy when they must not be visible to all LAN users.

Testing

```
python -m compileall -q backend
python -m pytest backend/tests -q
cd frontend
npm ci
npm run lint
npm run build
```

```
cd ..  
docker compose config
```

Troubleshooting

- Camera offline: inspect `/status`; check another program is not using the webcam, or verify RTSP access from the machine running FastAPI.
- Dashboard cannot connect: ensure `VITE_API_BASE_URL` points to the browser-visible backend address, then restart Vite.
- Docker on Windows: use RTSP. Run FastAPI locally for a physical Windows webcam.

Security and limitations

Set `AUTH_ENABLED=true` with a strong `JWT_SECRET`, `ADMIN_USERNAME`, and bcrypt `ADMIN_PASSWORD_HASH` for trusted-LAN administration. Mutation endpoints for zones and loitering rules require an admin bearer token when enabled; surveillance viewing remains available to the LAN. Browser tokens use local storage, so deploy only over HTTPS and use a VPN/reverse proxy for access control. SQLite is suitable for a single-node deployment; use PostgreSQL only when multi-process/multi-node scale requires it. Camera hardware, RTSP credentials, model accuracy, and alert tuning require deployment-specific testing.

Screenshots

Add dashboard and live-camera screenshots here for deployment documentation.